Linux, Git, and Reproducible Developer Workflows

Mick Cooney

15 October 2026

Table of Contents

1 What This Primer Covers **2**

2 What This Primer Is Not **3**

3 Assumed Background **3**

4 How to Read This Primer **3**

5 Linux Processes and the Shell Boundary **4**
5.1 What a Process Is 4
5.2 Signals and Shutdown 5
5.3 File Descriptors: Where Input and Output Go 5
5.4 Paths, Filesystems, and Mounts 6
5.5 Users, Groups, and Privilege 7
5.6 Environment Variables and Inheritance 8

6 How a Shell Turns Text Into a Process **8**
6.1 Command Lookup and Arguments 8
6.2 Quoting, Expansion, and Globbing 9
6.3 Pipelines, Redirection, and Exit Status 10
6.4 Diagnostic Discipline 11

7 Worked Example: Finding and Recovering a Failed Pipeline **12**

8 Git's Object and Repository Model **13**
8.1 What Git Stores 13
8.2 Working Tree, Index, and `HEAD` 14
8.3 Commits and References 15

9 Branches, Integration, and Collaboration **16**
9.1 Merge and Rebase 17
9.2 Review and Protected-Main Workflows 17

10 Recovery: Inspect Before You Repair **18**
10.1 Recovery Walkthrough: A Commit Hidden by Reference Movement 19
10.2 Reset, Restore, Clean, and Other Risk Boundaries 21
10.3 Reflogs and Worktrees 21

11 Reproducible Environments and Developer Lifecycle **22**
11.1 Declare Dependencies and Preserve Resolution 23
11.2 Isolate the Environment 24
11.3 Pin Tools and Images at the Right Boundary 25
11.4 Use Containers as Build Boundaries 26
11.5 Design CI as an Independent Recheck 27
11.6 Treat Generated Artefacts as Published Interfaces 28
11.7 Freshness, Validation, and the Stale-Output Failure 29

12 Review, Release, and Safe Lifecycle **30**
12.1 Repository Hygiene and Reviewable Changes 30
12.2 Release Workflow and Safe Lifecycle 31

13 Glossary **32**

14 Further Reading **33**

1 What This Primer Covers

Software work is a chain of state changes. A shell turns text into a process; the operating system decides what that process can see and do; Git records source state and collaboration history; a build environment turns source into artefacts; and review or release checks decide whether those artefacts are fit to publish. A failure at one boundary can look like a failure at another.

This primer develops the mental models beneath those boundaries. It covers Linux processes, filesystems, permissions, users, groups, environment variables, shell execution, pipelines, redirection, exit status, Git objects and references, branches, merges, rebases, reflogs, worktrees, reproducible environments, dependency state, containers as build boundaries, CI, and artefact validation. It uses short command sequences to make those models concrete and ends with three failure-and-recovery scenarios.

The emphasis is diagnosis and recovery. When a command fails, you should be able to state which layer owns the behaviour, gather evidence without destroying useful state, make the smallest justified repair, and verify the result.

2 What This Primer Is Not

This is not a Linux administration manual, a shell-language reference, a catalogue of Git commands, or a container-engine comparison. It does not teach kernel development, shell customisation, Kubernetes, coding-agent operation, or AI evaluation in depth. Use the containerisation primer for namespaces, cgroups, image layers, runtimes, and orchestration; the Oh My Zsh primer for Zsh configuration; and the Pi coding-agent primer for agent sessions and trust boundaries.

The commands and outputs shown in the worked examples are produced in disposable directories, disposable repositories, or documented build containers. They are not run against this repository's real history. Examples are designed to teach transferable reasoning, not to prescribe one project's only workflow.

3 Assumed Background

You should be comfortable opening a terminal, editing a text file, running a program, and reading a basic error message. You do not need to know Git's internal object types or Linux system calls in advance. The primer introduces those concepts when they become useful.

The main shell examples use Bash. Bash extensions are identified where they matter; portable shell scripts should follow the relevant POSIX rules instead of assuming that pipefail, arrays, or other Bash features exist. Product names, version-specific behaviour, and current hosting policy are not treated as timeless facts: verify them against the linked primary documentation before relying on them.

4 How to Read This Primer

Read the Linux and shell sections first if you are new to command-line systems. Read the Git sections next if your immediate problem is a branch, merge, rebase, or lost commit. Read the reproducibility and lifecycle sections when you need to make a result repeatable across machines or reviewable by someone else.

The recurring diagnostic loop is:

1. contain the operation and preserve the current state;
2. record the exact command, location, identity, inputs, output, and status;

3. inspect the layer that owns the observed behaviour;
4. change one hypothesis at a time;
5. verify both the repair and the absence of collateral damage.

Verified against primary sources for all sections; unverified claims are flagged inline.

5 Linux Processes and the Shell Boundary

Linux and a shell solve different parts of a command-line task. The kernel schedules processes, enforces access controls, and provides interfaces to files and devices. A shell reads command text, applies its language rules, and either performs an operation itself or asks the kernel to start another program. When a command behaves unexpectedly, identify which layer owns the behaviour before changing configuration. A shell expansion problem is not a permissions problem; a process that ignores a signal is not fixed by changing `PATH`.

5.1 What a Process Is

A process is a running instance of a program, with its own execution state and resources such as a virtual address space, open file descriptors, current working directory, credentials, and environment. The executable on disk is not the process: running the same executable twice creates distinct process instances. The kernel assigns each process a process ID (PID), which is useful for observation and signalling but is not a durable identity. PIDs can be reused after a process exits.

When a process starts another process, the new process is commonly described as its child and the starter as its parent. On Unix-like systems, a shell typically creates a child and then replaces that child's program image with the requested executable. The child inherits selected process attributes, including open descriptors, working directory, credentials, and environment, subject to changes made at launch. A parent can wait for a child and collect its termination status. This relationship explains why a command launched in a terminal usually ends when it finishes, while a background command can continue after the shell returns to its prompt. It does not guarantee that the process will survive logout: session management, terminal hangup handling, service managers, and shell behaviour also matter.

Inspect processes as a changing snapshot. `ps` reports process information; `pgrep` can select by name or other criteria, and `/proc/<pid>/` exposes Linux-specific process information while the process exists. A displayed command line may be truncated, altered by the program, or contain secrets. Do not treat it as an authoritative record of what the process has done. Establish which process you mean using more than a remembered PID, especially before sending a signal.

5.2 Signals and Shutdown

A signal is a small asynchronous notification delivered to a process or process group. Interactive terminals commonly send `SIGINT` when you press Ctrl+C. `SIGTERM` requests termination and gives a program a chance to clean up; `SIGKILL` forces termination and cannot be caught, blocked, or ignored. The distinction is operational: start with a graceful request, allow time for cleanup, inspect the process, and use a forced kill only when it is necessary. A process in uninterruptible kernel sleep may not disappear until the kernel operation completes, even after a kill request.

The `kill` command's name is historical: it sends a signal, not only a termination request. With no explicit signal, it normally sends `SIGTERM`. For example, `kill -TERM 4172` requests graceful termination of PID 4172. A negative PID can address a process group, so do not add or remove a minus sign casually. Permission checks apply: an unprivileged process generally may signal processes with matching credentials, while a privileged process has broader authority. See `kill(2)`, `signal(7)`, and the GNU Coreutils `kill` manual.

Do not begin by killing a process whose role you have not identified. First inspect its owner, parent, command, and relevant logs. A process may hold an unsaved edit, perform a database transaction, or supervise other processes. For a service, use the service manager's stop operation so that the configured shutdown path runs. After sending `SIGTERM`, check whether it exits; escalation is a decision based on observed state, not a reflex.

5.3 File Descriptors: Where Input and Output Go

Programs normally interact with input and output through file descriptors, small integers maintained by the kernel for an individual process. By convention descriptor 0 is standard input, 1 is standard output, and 2 is standard error.

These descriptors may refer to a terminal, regular file, pipe, socket, or another kernel object. They are conventions rather than special streams: a program can redirect or close them, and a child can inherit them.

This model explains redirection and pipelines. `command >result.txt` arranges for standard output to refer to the file before the command runs. `command 2>errors.txt` does the same for standard error. `command >all.txt 2>&1` first directs descriptor 1 to the file, then makes descriptor 2 refer to the same destination. Order matters: `command 2>&1 >all.txt` duplicates the original standard output to standard error before changing standard output, so errors may remain on the terminal. The shell, not the program, opens the redirection target. If opening it fails, the program may never start.

A pipe connects one process's standard output to another process's standard input. It does not automatically carry standard error. This distinction is useful when a pipeline appears to lose diagnostics: the error messages may still be on the terminal because descriptor 2 was never connected. Bash documents redirections and pipelines in its reference manual and pipelines section; descriptor duplication is also described by `dup(2)`.

5.4 Paths, Filesystems, and Mounts

A path is a name interpreted from a starting point. An absolute path begins at `/`, the process's filesystem root. A relative path begins at the process's current working directory. `.` names the current directory and `..` its parent in the path view available to that process. The shell maintains a current directory for its own commands; a child process inherits the directory at launch. Changing directory in a child cannot change the parent's working directory, which is why `cd` must normally be a shell built-in.

Linux presents filesystems through one directory tree. A mount attaches a filesystem at a directory, making that filesystem's contents visible below the mount point. The path `/home/sam/data` says where a file is named in the current mount view; it does not reveal which physical device or remote filesystem supplies it. The same path can resolve differently in different mount namespaces, containers, chroots, or service configurations. `findmnt` reports mount relationships, `df` reports capacity for a path's filesystem, and `stat` reports metadata for a path. Use `findmnt --target /some/path` or `df -h /some/path` to ask about the actual location instead of assuming that directory names imply storage

boundaries. The kernel's path resolution and mount interfaces are described in `path_resolution(7)` and `mount_namespaces(7)`.

Permissions are checked against the object reached through path resolution. Ordinary Unix mode bits distinguish an owning user, an owning group, and other users, with read, write, and execute/search bits for each class. On a regular file, execute permits running it subject to other checks; on a directory, execute means search permission — traversing or looking up names inside it. Reading a file may therefore fail because you lack search permission on one of its parent directories. Writing a file may require write permission on the containing directory if you need to create, remove, or rename a name. This is why `chmod` on the file is not a universal repair for "permission denied".

Use `id` to inspect your effective identity and groups, `namei -l /path/to/file` to inspect permissions along a path, and `stat` or `ls -l` to inspect an object. A leading `d` in the mode shown by `ls -l` means directory; the remaining mode characters describe the three permission classes. ACLs, mandatory access controls such as SELinux, mount flags, capabilities, and read-only filesystems can add constraints beyond those mode bits. When basic permissions look correct, check those layers before applying broad changes such as recursive ownership updates. The kernel's documented permission check is summarised in `path_resolution(7)`; see also `chmod(1)` and `acl(5)`.

5.5 Users, Groups, and Privilege

The kernel associates credentials with a process, including a real and effective user ID, group IDs, and supplementary groups. A program may have an effective identity different from the user who launched it, for example through a set-user-ID executable or a privileged service. `id` describes your current shell's identity; it does not necessarily describe a service or another process. To inspect a process, use tools such as `ps -o pid,ppid,user,group,stat,args -p <pid>` and, where permissions allow, inspect `/proc/<pid>/status`.

`sudo` runs an authorised command with credentials granted by policy. It does not make a questionable command safe, and it does not repair a misunderstanding about which path or account is involved. Before using elevated privileges, verify the command, target, and working directory. Prefer a narrow privileged operation over launching an entire interactive shell as root. Avoid `sudo` with shell pipelines unless you understand which side receives elevated credentials:

in `producer | sudo consumer`, only `consumer` is elevated. The pipe itself connects processes; it does not transfer one process's credentials to another.

5.6 Environment Variables and Inheritance

An environment is a set of name-value strings passed to a newly executed program. In a shell, an ordinary assignment such as `MODE=debug` changes a shell variable. `export MODE` marks it for inclusion in the environment of subsequently launched children. Thus `MODE=debug command` sets the variable for that command's environment without permanently changing the shell's exported value. A child cannot change its parent's environment after launch; it can only affect processes it later starts.

Environment variables commonly configure programs: `HOME` names the user's home directory, `LANG` and related variables influence locale behaviour, and `PATH` helps locate executable commands. The values are not inherently trustworthy. A service may receive a deliberately small environment; a shell may inherit variables from a terminal, login manager, container runtime, or script. `env` displays the current environment, but output can include credentials or tokens. Inspect selectively and redact before sharing. The interfaces are documented by `environ(7)` and the Bash manual's environment section.

6 How a Shell Turns Text Into a Process

The shell section follows one command from text to execution and status. First establish how a name becomes a program and how text becomes arguments; then examine quoting, expansion, pipes, redirection, and the evidence needed when a stage fails.

6.1 Command Lookup and Arguments

The shell does not pass a typed command line verbatim to a program. It parses shell syntax, performs expansions, applies redirections, and constructs an argument vector. The program receives arguments as separate strings: argument zero conventionally names the invoked program, followed by the arguments you supplied. Spaces separate words unless quoting or escaping keeps them together. Consequently, a path containing spaces must be quoted when you intend it to be one argument.

For a command name without a slash, shells such as Bash search functions, built-ins, and executable files found through `PATH` according to shell rules. An explicit path such as `./script` or `/usr/bin/tool` bypasses the `PATH` search. `type -a command` in Bash can show whether a name resolves to an alias, function, builtin, or one or more executable files; `command -v command` gives a concise resolution. `which` may be an external utility and can disagree with shell lookup when aliases or functions are involved. If the wrong version runs, inspect `type -a`, the ordered `PATH`, and the executable's identity with `command -v` and `ls -l` before reinstalling anything.

The order of directories in `PATH` matters. The first matching executable normally wins, so a project shim or user-level tool directory can take precedence over a system installation. An empty component in `PATH` can mean the current directory in some shell and system configurations; including `.` can make command execution depend on the current directory and is a security hazard. Prefer explicit paths for local scripts and avoid adding writable, untrusted directories ahead of trusted system locations. See the Bash command search documentation and `execve(2)`.

6.2 Quoting, Expansion, and Globbing

Quoting controls which characters the shell treats as syntax. Single quotes preserve every enclosed character literally, except that a single quote cannot occur inside a single-quoted string. Double quotes preserve most characters while still allowing parameter expansion, command substitution, and selected backslash behaviour. A backslash can quote the next character outside single quotes. Prefer quoting a variable expansion when it represents one intended argument: `tool "$filename"`. Without quotes, spaces split the expanded value into multiple words and wildcard characters may expand against matching filenames. Bash's quoting and word-splitting sections define these rules.

For the examples here, the relevant expansion order is that the shell expands parameters such as `$HOME`, substitutes command output in `$(command)`, performs word splitting on eligible unquoted results, expands filename patterns such as `*.log`, and then removes quoting syntax. Bash also has earlier or interleaved stages, including brace, tilde, arithmetic, and process substitution; consult its complete expansion rules when exact ordering matters. A quoted wildcard such as `'*.log'` is passed literally; an unquoted `*.log` usually becomes one argument

per matching name. If no name matches, behaviour depends on the shell and its options. Do not assume that a command receives the text you see in the script: inspect the quoted form and, for a safe diagnostic, use `printf '<%s>\n' "$value"` to make argument boundaries visible.

Command substitution captures standard output and removes trailing newline characters. It is useful for values, but it obscures status handling if used casually. For example, `value=$(some_command)` runs the command in a substitution environment; the assignment's status generally reflects the substitution's status in Bash, but more complicated declarations or surrounding commands can mask it. Check failures explicitly when the result matters. Avoid `eval`: it asks the shell to parse a second time and can turn data into executable syntax.

6.3 Pipelines, Redirection, and Exit Status

In `first | second`, the shell starts commands connected by a pipe. Each stage normally runs in a separate process environment, although shell details govern built-ins and pipeline subshells. The pipeline's status is usually the status of its final command. This can hide an earlier failure: if a producer fails but a consumer successfully reads an empty stream, the overall status may be zero. Bash's `set -o pipefail` changes the pipeline status to the rightmost non-zero status, allowing an earlier stage's failure to be noticed. This option is Bash-specific; POSIX shell scripts should not assume it exists. Consult Bash's pipeline exit-status rules and the `set` builtin.

`$?` expands to the status of the most recently completed foreground pipeline. Zero conventionally indicates success; a non-zero value indicates failure or another documented condition. It is not a universal error vocabulary: each program defines its own statuses, and shells may encode signal termination using a value such as 128 plus the signal number. Capture the status immediately if you need it, because the next command replaces `$?`. In a compound operation, use explicit conditionals or store the value rather than relying on a later prompt indicator.

Redirection can change what evidence you see. `>` creates or truncates a file; `>>` appends; `<` supplies input; `2>` captures diagnostics; `2>&1` duplicates standard error to the current standard output destination. A pipeline generally connects only standard output. To send both streams through a pipe in Bash, use `2>&1 |` or the Bash shorthand `|&`, while remembering that combining streams loses

the distinction between data and diagnostics. Keep diagnostics separate when a later stage expects structured or clean data.

6.4 Diagnostic Discipline

Troubleshooting works best as a sequence of small, reversible observations. First record the exact command, working directory, identity, relevant environment, and complete error output. Then state what was expected and what happened. Avoid paraphrasing an error before preserving it; a single word such as “permission”, “not found”, or “busy” can point to a different layer.

Next, split the command into boundaries. Confirm what the shell resolved (`type -a`), what arguments it formed (careful `printf` or shell tracing in a safe environment), which files and mounts the path refers to (`stat`, `findmnt`, `df`), and which user and groups apply (`id`). For a running program, check the actual process, parent, status, and logs rather than inferring them from a prompt. Observe exit statuses one stage at a time. Change one hypothesis at a time and rerun the smallest command that can disprove it.

Shell tracing (`set -x` in Bash) prints commands after expansion. It can expose unexpected splitting or variable values, but it can also disclose passwords, tokens, and private paths in logs. Enable it only around a narrow, non-sensitive operation, and turn it off with `set +x`. For system calls, `strace` can show file opens, process creation, and failed kernel calls, but it produces detailed output and may expose data; use it only when process-level evidence is needed and access is appropriate. `man` pages, `--help` output, and official documentation should be checked for the exact program and version rather than trusting a similar command’s conventions.

7 Worked Example: Finding and Recovering a Failed Pipeline

Suppose a report is meant to select lines containing `READY`, sort them, and write the result to a file. A producer occasionally fails because its input file is absent. Without pipeline failure handling, `sort` can receive an empty stream and successfully write an empty result, making the whole command appear successful. The following Bash session reproduces the problem and repairs it in a newly created temporary directory. The output shown is from that isolated run; no repository files or history are involved.

```
work=$(mktemp -d)
cd "$work"
printf 'READY: one\nWAIT: two\nREADY: three\n' > events.log

grep 'READY' missing.log | sort > report.txt
printf 'status=%s\n' "$?"
```

Expected output:

```
grep: missing.log: No such file or directory
status=0
```

The non-zero `grep` failure was hidden because `sort`, the final pipeline stage, completed successfully. The shell created or truncated `report.txt` before starting the pipeline, so the output file exists but contains no selected records. That is a meaningful partial side effect: an empty file is not proof that the input contained no matches.

First make pipeline status report earlier failures. Then rerun the original command. The expected status is non-zero, even though `sort` itself can process an empty stream:

```
set -o pipefail
grep 'READY' missing.log | sort > report.txt
printf 'status=%s\n' "$?"
```

Expected output:

```
grep: missing.log: No such file or directory
status=2
```

The status value comes from this `grep` implementation and invocation; the important diagnostic is that the pipeline failed. Now check the path and recover using the existing input. The check prevents an absent input from being confused with a valid file containing no matching lines:

```
test -r events.log && grep 'READY' events.log | sort > report.txt
status=$?
printf 'status=%s\n' "$status"
printf '%s\n' '--- report ---'
cat report.txt
```

Expected output:

```
status=0
--- report ---
READY: one
READY: three
```

The corrected command verifies that the intended input exists and is readable, then uses `pipefail` so a producer or sorting failure reaches the caller. For a reusable script, handle each possible status deliberately: a missing input is an error, while no matching lines may be an expected result depending on the task. Preserve the original file until the replacement has been produced and checked when overwriting it would destroy useful data. A more careful production workflow often writes to a temporary output in the same filesystem, validates it, and renames it into place only after success. The key habit is to ask what each stage did, which status survived, and which output side effects occurred before declaring recovery complete.

8 Git’s Object and Repository Model

Git’s stored objects and movable references explain the working tree, commits, collaboration, and recovery that follow. Start with the object graph, then separate the working tree and index from `HEAD`, before examining commits and the names that make them reachable.

8.1 What Git Stores

Git is a content-addressed database with a versioned directory tree and a set of references that name commits. This model explains both ordinary collaboration and why an apparently lost commit can often be recovered. Git does not store a sequence of full directory copies, nor is a branch a container holding files. Most commands create, inspect, or move objects and references; the working tree is a checked-out view assembled from them.

Git hashes the content and type of each object to produce its object ID. Repositories created with SHA-1 use 40 hexadecimal characters; repositories configured for SHA-256 use a different object format and longer identifiers. Treat IDs as opaque names and use the full value when precision matters. A commit records its root tree, parent commit or commits, author and committer metadata, timestamp, and message. A tree records names, modes, and object IDs for files and subdirectories. A blob stores file content. An annotated tag is another object

that points to an object, usually a commit, and can include a message and signature. These relationships form a directed graph, usually with commits pointing backwards to parents.

Identical blobs or trees can be shared by many commits. A commit therefore describes a snapshot by referring to trees, not a patch applied to the preceding commit. A diff is a useful comparison Git calculates between snapshots; it is not the underlying storage format. A commit with one parent extends a line of history. A merge commit has two or more parents and records the point where histories were joined. Root commits have no parents.

Most objects are initially loose files under `.git/objects`, but Git can pack them for space and speed. Do not edit these files directly. Commands such as `git cat-file`, `git ls-tree`, and `git show` provide supported ways to inspect object contents and relationships. The official Git object model documentation describes the object types and how trees and commits refer to them.

8.2 Working Tree, Index, and `HEAD`

The working tree is the set of checked-out paths on disk. The index, also called the staging area, is the proposed content for the next commit. `HEAD` usually names the current branch; the branch reference in turn names its current tip. The three are related, but they are not interchangeable. A file can have one version in the current commit, another version staged in the index, and a third version in the working tree.

`git status` summarises differences across these boundaries. “Changes to be committed” means the index differs from `HEAD`. “Changes not staged for commit” means the working tree differs from the index. “Untracked files” have no indexed or committed version. A clean status means those comparisons reveal no changes; it does not prove that the current branch is correct, that a commit is reachable from the expected remote, or that ignored files are safe to delete.

`git add` copies the current file content into the index. It does not permanently mark the path for all future changes: editing the file again creates a working-tree difference that must be staged separately. `git diff` compares the working tree with the index; `git diff --staged` compares the index with `HEAD`; `git diff HEAD` compares the working tree with the current commit. Review the staged diff immediately before committing because it is the actual proposed snapshot.

The index also tracks file modes and merge state. During a conflict it can hold multiple unmerged entries for a path, rather than one final version. `git status` reports these paths. Resolve a conflict in the working tree, inspect the result, then stage the resolved path to replace the unmerged index entries. A merge is not complete until every conflict is resolved and the operation is committed or deliberately aborted.

In a normal branch checkout, `HEAD` is a symbolic reference such as `ref: refs/heads/main`. In detached-`HEAD` state, it names a commit directly. Detached state is useful for inspection and experiments, but a new commit may not be reachable by a named branch after checkout elsewhere. If you need to keep it, create a branch at that commit before moving away.

8.3 Commits and References

A commit freezes the index as a tree and records its parent or parents. It does not include unstaged changes. Good commits describe a coherent change, use a useful message, and keep generated artefacts and secrets out unless the project explicitly requires them. Before committing, inspect `git status`, `git diff`, and `git diff --staged`; after committing, inspect the new commit with `git show --stat --oneline HEAD`.

A reference is a human-readable name that resolves to an object. Branches, tags, and remote-tracking branches are references. A branch such as `main` is a movable pointer to a commit. Creating a branch is cheap because it creates a name, not a copy of the repository. `HEAD` usually follows the checked-out local branch. A lightweight tag is a reference directly to an object; an annotated tag is a Git object that can carry a message and signature and is preferred for release labels.

References can move. Creating a commit advances the checked-out branch. `git reset` moves a branch reference and may also change the index or working tree, depending on its mode. `git commit --amend` creates a replacement commit and moves the branch to it. These operations do not rewrite objects in place; the old commits may remain in the object database for a time, but if no reference reaches them they become unreachable and eventually eligible for pruning. This distinction makes recovery possible, but it is not a reason to treat unreachable data as permanent backup.

The Git reference documentation explains references, symbolic references, and tags. The Git glossary is useful when terms such as “reachable”, “tracking branch”, “index”, and “refspec” appear in command output or documentation.

9 Branches, Integration, and Collaboration

Local branches are names in your repository. A remote is a configured name and URL, commonly `origin`, used to fetch from and push to another repository. A remote-tracking branch such as `origin/main` records the last fetched view of a remote branch. It is not a live connection and does not update until a fetch or another network operation updates it. `git remote -v` shows configured URLs; `git branch -vv` shows local branches and their upstream relationships.

`git fetch` obtains objects and updates remote-tracking references without changing the current branch, index, or working tree. It is a good first step before comparing local and remote history. `git pull` is broadly a fetch followed by integration, usually merge or rebase according to configuration. Since the integration may alter local history or produce conflicts, separate `fetch` and `merge`/`rebase` steps are easier to inspect when the state is uncertain.

A local branch often has an upstream, for example `origin/main`. `git status` can then report commits ahead of or behind that upstream, but only relative to the last fetched information. `git push` sends reachable objects and asks the remote to update a reference. A non-fast-forward rejection means the remote reference no longer has the local tip as an ancestor; fetch and inspect the divergence before deciding how to integrate it. Do not force-push to overcome a rejection without understanding who else uses the branch.

For shared work, prefer short-lived topic branches, explicit review, and a protected default branch. A protected-main workflow keeps `main` available as an agreed integration point: changes arrive through reviewed pull requests, required checks, and controlled merges. Branch protection is server policy, not a property enforced by local Git. Local hooks can help catch mistakes, but a developer can bypass them and they cannot replace server-side permissions and required checks. Consult the hosting provider’s current documentation for its exact rule names and behaviour; Git itself defines the repository operations, not a universal pull-request process.

9.1 Merge and Rebase

A merge integrates histories by finding a common ancestor and combining the changes since that point. If one side is already an ancestor of the other, Git can fast-forward the target reference without creating a merge commit. Otherwise, a merge commit records both parent histories. Merging preserves the fact that two lines of work were joined. It does not change the commits that already exist.

A rebase selects commits from one line, replays their changes onto another base, and creates new commits with new object IDs. The original commits remain distinct objects; the rebased commits have different identities because their parent and commit data changed. Rebasing can make a topic branch appear linear, but it changes history and can make review or collaboration confusing if others built work on the original commits. Rebase commits that only you control; coordinate before rewriting shared branches.

Before either operation, check the branch and working tree. Save or commit work in progress, fetch if remote state matters, inspect `git log --graph --oneline --decorate --all`, and choose the intended base explicitly. During a conflict, read `git status` and inspect each path in context. Conflict markers show alternative text, not a guaranteed correct answer. The right resolution may combine both sides, retain one side, or redesign the change. Run relevant checks and inspect the complete diff after staging the resolution.

For a conflicted merge, `git merge --abort` attempts to return to the pre-merge state. For a conflicted rebase, `git rebase --abort` returns to the branch state from before that rebase. An abort is a deliberate state-changing action: first inspect uncommitted work and understand whether the operation began with a clean tree. If an abort fails or the result looks wrong, stop and inspect status, reflog, and saved work before trying a reset. `git rebase --continue` proceeds after conflicts are resolved and staged; `git rebase --skip` omits the current patch and can discard the change it represents, so use it only when that omission is intentional.

9.2 Review and Protected-Main Workflows

Review works best when the proposed change is explicit and small enough to reason about. On a topic branch, inspect the full difference from its base, not just the most recent commit: `git diff origin/main...HEAD` compares the branch's

merge base with its tip. Confirm the target base, file list, generated outputs, and test or build evidence. A reviewer should be able to connect each change to the issue or design decision and see how important behaviour was checked.

After review comments, make focused follow-up commits or amend the topic history if that branch is private to the contributor and the project prefers a clean series. If reviewers or collaborators have based work on the branch, rewriting it can disrupt them. Before updating a pull request after rebasing, inspect the range being replaced and use `--force-with-lease` only when rewriting is intended and the lease confirms the remote ref has not moved unexpectedly. `--force-with-lease` reduces one class of overwrite risk; it does not make a careless force-push safe.

Once checks and review are complete, merge according to project policy. A merge commit retains branch topology; a squash merge creates one combined commit on the target; a rebase merge replays commits onto the target. Hosting services may implement these choices with policies that differ in detail. A local branch's post-merge relationship to the remote depends on which method was used. Fetch, inspect the remote-tracking reference, then update local branches deliberately. Delete a topic branch only after verifying the merged change is present and no remaining work depends on that branch.

10 Recovery: Inspect Before You Repair

When something looks lost, first preserve evidence. Stop running commands that move references or overwrite files. Do not start with `reset --hard`, `clean`, `gc`, pruning, or another cleanup operation. Record the current directory and repository, then inspect:

```
git status --short --branch
git branch -avv
git log --graph --decorate --oneline --all -30
git reflog --all --date=iso
git stash list
```

These commands answer different questions. `status` reveals tracked and untracked working-tree changes. Branch listings show which names still point to commits and their recorded upstreams. The graph shows reachable history. Reflogs record recent local movements of references such as `HEAD` and branches. Stashes may contain saved tracked changes and, depending on how they were created, untracked files. None of these views alone is a complete backup.

The worktree itself may contain valuable uncommitted edits. Copy important files or create a safe patch before recovery operations, taking care not to overwrite the only copy. A patch does not preserve every property, such as untracked files, file modes in every workflow, or repository metadata. For consequential recovery, make a filesystem copy or a bundle when possible, and verify that it can be read. Do not copy an actively changing repository as though the copy were automatically consistent.

A commit is recoverable while its object remains available, even if no branch names it. The reflog is often the easiest route to an unreachable commit, but reflog entries expire according to configuration and unreachable objects can later be pruned. A remote, another clone, a tag, a worktree, or a backup may retain the commit even after local reflog evidence is gone. If a commit is absent from all refs and object storage has pruned it, Git cannot recreate its content from the commit ID alone.

10.1 Recovery Walkthrough: A Commit Hidden by Reference Movement

This example uses a disposable repository. It simulates two branch movements after a useful commit, leaving the commit absent from the visible branch tips. The reflog identifies the prior commit; inspection confirms its content; a new branch preserves it. The example uses hard resets only in this throwaway repository, with no uncommitted changes. Do not copy those reset commands into a working repository as a generic recovery recipe.

Create an isolated repository and make a baseline commit followed by a useful commit.

```
sandbox_dir=$(mktemp -d /tmp/git-recovery.XXXXXX)
mkdir "$sandbox_dir/repo"
git -C "$sandbox_dir/repo" init -q -b main
git -C "$sandbox_dir/repo" config user.name 'Primer Example'
git -C "$sandbox_dir/repo" config user.email 'primer@example.invalid'
printf 'baseline\n' > "$sandbox_dir/repo/notes.txt"
git -C "$sandbox_dir/repo" add notes.txt
git -C "$sandbox_dir/repo" commit -qm 'Add baseline note'
printf 'drafted on topic\n' >> "$sandbox_dir/repo/notes.txt"
git -C "$sandbox_dir/repo" commit -qam 'Add a useful draft'
```

Now move the `review` branch and then `main` back one commit each. This deliberately contrived setup removes the useful commit from both branch tips and leaves no tag naming it. The reflog remains the evidence source. The branch names point to the baseline, and the working tree is clean.

```
git -C "$sandbox_dir/repo" checkout -qb review
git -C "$sandbox_dir/repo" reset --hard HEAD~1
git -C "$sandbox_dir/repo" checkout -q main
git -C "$sandbox_dir/repo" reset --hard HEAD~1
git -C "$sandbox_dir/repo" status --short --branch
git -C "$sandbox_dir/repo" show-ref --head
git -C "$sandbox_dir/repo" reflog -6 --date=iso
```

Captured output from this run (the object IDs are specific to the demonstration run):

```
## main
373f2c16884893dfa83594754a9df157ff9a7cc8 HEAD
373f2c16884893dfa83594754a9df157ff9a7cc8 refs/heads/main
373f2c16884893dfa83594754a9df157ff9a7cc8 refs/heads/review
373f2c1 HEAD@{2026-10-15 22:54:36 +0100}: reset: moving to HEAD~1
0a7e973 HEAD@{2026-10-15 22:54:36 +0100}: checkout: moving from review to main
373f2c1 HEAD@{2026-10-15 22:54:36 +0100}: reset: moving to HEAD~1
0a7e973 HEAD@{2026-10-15 22:54:36 +0100}: checkout: moving from main to review
0a7e973 HEAD@{2026-10-15 22:54:36 +0100}: commit: Add a useful draft
373f2c1 HEAD@{2026-10-15 22:54:36 +0100}: commit (initial): Add baseline note
```

Do not choose a reflog entry only because its position looks plausible. Resolve the candidate and inspect its metadata and patch. Here, `HEAD@{1}` names the useful commit in the captured state, but reflog selectors are relative and change as new operations occur. Resolve the candidate to a full object ID before creating the rescue branch, so later commands cannot change what the selector means.

```
candidate=$(git -C "$sandbox_dir/repo" rev-parse 'HEAD@{1}')
git -C "$sandbox_dir/repo" show --stat --oneline "$candidate"
git -C "$sandbox_dir/repo" show "$candidate" --
```

The inspection reports `Add a useful draft` and one added line in `notes.txt`. If that is the desired commit, preserve it with a new reference before doing further work. `git branch` creates the branch without changing the checked-out branch or working tree.

```
git -C "$sandbox_dir/repo" branch rescue/draft "$candidate"
git -C "$sandbox_dir/repo" log --oneline --decorate --all -4
git -C "$sandbox_dir/repo" show rescue/draft:notes.txt
```

The captured verification showed `rescue/draft` pointing at `0a7e973`, and the file contained both `baseline` and `drafted on topic`. The recovery is complete in the narrow sense that a branch now keeps the commit reachable. Decide separately how to integrate it: compare it with the intended target, create a new branch from the right base, and cherry-pick or merge only after confirming the desired history. Keep the rescue branch until the integrated result is reviewed and backed up. Do not delete the temporary repository until you no longer need the evidence; it is under `$sandbox_dir/repo` until you remove it.

This approach also applies after an accidental rebase or branch switch. Rebase commonly leaves old commits discoverable through reflogs, although selectors

and exact reflog entries depend on the commands run. Inspect `git reflog --all`, find the old tip, use `git show <candidate>` to verify it, then create a rescue branch such as `git branch rescue/old-topic <candidate>`. Reflog recovery restores a reference to existing objects; it does not reconstruct edits that were never committed or saved.

10.2 Reset, Restore, Clean, and Other Risk Boundaries

Git commands that sound like "undo" act on different layers. `git restore <path>` can replace working-tree content from the index; `git restore --staged <path>` can remove a path from the index using `HEAD` as the default source. These operations can discard changes at the layer they replace. Inspect the source and destination with `git status` and diffs first. To unstage a file while retaining working-tree edits, `git restore --staged <path>` is usually clearer than resetting the entire branch.

`git reset --soft <commit>` moves the current branch while leaving the index and working tree in place. `git reset --mixed <commit>` (the default) moves the branch and resets the index while retaining working-tree changes. `git reset --hard <commit>` moves the branch, resets the index, and overwrites tracked working-tree paths; untracked files are not a reliable safety net and can be removed by related commands. `git clean` removes untracked files and directories, with force and directory options making the scope broader. Preview clean candidates with `git clean -nd` before any removal, and check ignored files separately if they may contain valuable local data.

Before a destructive operation, identify the exact repository, branch, paths, and target commit. Run status and inspect the relevant diffs. Preserve uncommitted work and establish a recovery reference for commits that may become unreachable. Prefer a targeted operation with a clear source to a broad reset. If the intended state is uncertain, stop after inspection. The Git reset documentation, restore documentation, and clean documentation define the affected layers and options.

10.3 Reflogs and Worktrees

The reflog records recent updates to references in a local repository. `HEAD` has a reflog in typical repositories, and branch references may have their own. `git`

`reflog show --all` can reveal a commit reached before a reset, rebase, checkout, or amend. Reflogs are local records, not part of the commit graph shared by fetch and push. A fresh clone does not normally inherit another clone's reflog. Expiry settings and garbage collection mean reflogs are temporary recovery aids, not archives.

`git worktree` creates additional working directories attached to one repository, usually with a different branch checked out in each. It is useful when you need to compare or test branches without repeatedly changing the primary working tree. Worktrees share most object storage and references, but have separate working directories and per-worktree `HEAD` state. A branch normally cannot be checked out in two worktrees at once, which prevents accidental concurrent edits to one branch tip. `git worktree list` shows attached worktrees; `git worktree remove` removes one, so first inspect its uncommitted and untracked files. Worktrees are not independent backups because they share the same repository objects and many references.

The Git reflog documentation and worktree documentation cover retention and management details. For long-lived recovery, use an independent clone, remote, bundle, or filesystem backup in addition to local reflogs.

11 Reproducible Environments and Developer Lifecycle

A reproducible build begins with a clear answer to four questions: which source revision is being built, which tools and dependencies are inputs, what command transforms them, and which outputs count as the result? A version-controlled build recipe is the executable part of that contract. A README can explain the workflow, but if its command disagrees with the actual build file, the build file and observed behaviour are stronger evidence.

"Reproducible" has more than one useful strength. At the weakest level, a new contributor can follow documented steps and obtain a working result. A stronger claim is that specified inputs reliably produce functionally equivalent outputs. The strict reproducible-build definition is bit-for-bit identity when source, build environment, and instructions are the same; cryptographic hashes can compare those output bytes. State which level matters. A PDF containing a generation timestamp may be useful and repeatable without being byte-identical. Reprodu-

cible Builds defines the strict form and its inputs.

Record the inputs that can affect the outcome: source commit, dependency versions, system libraries, compiler or interpreter, build flags, environment variables, locale, time zone, network-fetched data, and relevant external services. Then reduce that list. An environment variable that does not affect output need not become a formal input; one that selects a locale or endpoint may be essential. Hidden defaults are still inputs, even when they are not written in a manifest.

Use the project's normal build entry point. For a primer in this repository, that means the directory's `Justfile` and shared recipes in `_shared/common.just`, not a command reconstructed from memory. The recipe chooses a pinned Pandoc build image, passes title and date metadata, mounts the source tree, and defines outputs. The root `Justfile` provides selected entry points, while the project recipe controls the complete build. The repository's contribution guide documents which recipes apply and why generated HTML and PDF are committed alongside their source.

11.1 Declare Dependencies and Preserve Resolution

A dependency declaration expresses what the project needs; a lockfile records a particular resolution of those needs. These answer different questions. A declaration such as "any compatible version from this major release" can be appropriate for a library, but it leaves a package manager room to select different transitive dependencies over time. A lockfile can capture the resolved versions and often their integrity hashes, making installation more repeatable. Neither file records every operating-system library, compiler, or network service used by the build.

Use the dependency mechanism native to the ecosystem and commit its lockfile when the project's tooling supports one. Keep the declaration and lockfile consistent, and update them through the package manager rather than hand-editing resolved graphs. Review lockfile changes as code: ask which top-level request caused a large transitive shift, whether a package source changed, and whether checks still exercise the intended versions. A lockfile is input data, not noise to regenerate on every machine.

Distinguish application dependencies from developer and build dependencies. Test runners, formatters, documentation generators, code generators, and

compilers can change published output or gate acceptance even if they are absent from the runtime program. Declare them in the appropriate development or build group. If a tool is installed globally or invoked through an unpinned `npx`/`pipx`/shell download command, it is still a dependency; it is only an undeclared one.

Hashes strengthen a lock by checking that downloaded package bytes match the reviewed resolution. They do not establish that the package is benign, maintained, or suitable. Verify the source, licence, release provenance, and install-time behaviour according to the project's risk. Private credentials belong in a secret store or local environment, never in a dependency file or committed lockfile.

A dependency update is a controlled change: update the declared range only when intent changes, refresh the lock through the ecosystem's resolver, inspect the diff, and run the project checks with that resolution. If a lockfile format changes or a resolver is upgraded, review the effect separately from ordinary dependency updates. A clean install from a fresh environment is stronger evidence than a successful install over an old environment that may contain undeclared packages.

11.2 Isolate the Environment

Isolation limits accidental interaction between projects and makes the environment easier to reconstruct. Language-specific virtual environments, package-manager-managed toolchains, and containers solve related but different problems. A virtual environment separates one language's packages from system or neighbouring project packages. A version manager selects a runtime or compiler. A container packages a user space and its tools. None automatically pins every input or makes untrusted code safe.

Build in a disposable environment when practical. Start from a clean checkout, create or restore the project environment from its declarations and lock, then run the documented build. Avoid relying on packages left in a global interpreter, shell customisation, user configuration, or credentials inherited from your login session. Pass only the environment variables the build needs. Keep caches outside the project's declared inputs and make the build succeed when a cache is empty; a cache may accelerate reconstruction but must not be required for correctness.

Separate generated files from source and temporary state. Put disposable build directories in an ignored location such as `.build/`; keep credentials and local overrides out of version control; and do not let an ignored file silently become a required dependency. A useful diagnostic is a clean checkout in a new directory or worktree. If the build passes only in the long-lived working directory, inspect ignored files, environment variables, and globally installed tools before changing the recipe.

Isolation is also a safety boundary, but not a complete one. A container with the project directory mounted read-write can still change those files, and a process with network access can still send data outward. Run unfamiliar generators and third-party build scripts with the smallest necessary filesystem and credential access. Container implementation and operating-system isolation details belong in the containerisation primer.

11.3 Pin Tools and Images at the Right Boundary

Pinning gives a build a stable name for each important tool input. Pin a runtime or compiler version in the project's supported version file; pin package versions in its lockfile; pin CI actions and reusable workflows to reviewed immutable revisions where the platform permits it; and use an image digest when the exact container image matters. A human-readable tag or version label is convenient but may point to changed bytes later. A digest identifies content, while a tag is a label that maintainers may move. Docker's image reference documentation describes digest references.

Pinning everything forever is not the objective. It creates a maintenance obligation: someone must check for fixes, assess changes, refresh the pin, and rerun the build. Choose a policy that matches the consequence of drift. A stable release branch may require exact tool and image inputs. A development tool may accept a supported version range, with a scheduled update and a lockfile recording the current resolution. Record where each pin is maintained and how to update it.

Prefer one documented build boundary that owns the compiler, renderer, and system packages needed to create the output. This repository uses a shared Pandoc container so a host installation of XeLaTeX or a particular Pandoc release does not become an invisible prerequisite. The root `Justfile` builds or fetches shared images through `just setup`; project Justfiles import the shared recipes.

This boundary improves consistency, but the source mount, runtime, host kernel, architecture, and external services can still affect a build. Describe it as a controlled environment, not as proof of complete determinism.

Updates should be deliberate and observable. Confirm an upstream release or digest from its primary source, update the central declaration, rebuild a representative project, and inspect output changes. Avoid copying the same pin into many files; centralised declarations reduce drift. If architecture-specific manifests or platform variants are involved, identify which one the pin selects and whether the project expects the same platform everywhere.

11.4 Use Containers as Build Boundaries

A container is useful here because it packages build tools and system libraries behind a repeatable command. The build recipe should make the boundary visible: image, mounted source path, working directory, environment, command, and output location. Developers should invoke the same recipe locally and in CI wherever possible. Parallel shell commands that "happen to work" weaken the contract because they create separate build definitions.

Treat the image as an input, not as a magic guarantee. Pin or otherwise control its source, document how it is updated, and make image creation reproducible enough for the project's goal. Keep credentials out of image layers and avoid writing generated outputs into an unexpected host path. A read-only source mount is ideal for verification jobs that should not modify tracked files; a build job that must write outputs needs an explicit output directory.

The boundary should be narrow and boring. Install only the tools required for that task, use a non-root user where supported, and do not mount the container engine socket unless the build genuinely requires it. An image build that fetches unpinned packages at a later step may drift even when its base image is pinned. Capture that remaining variability in the contract or remove it by using a package snapshot, checked-in lock, or verified artefact.

Do not duplicate the container primer's account of namespaces, image layers, runtimes, rootless modes, or orchestration. Link to that primer when the question becomes "what isolation does this container provide?" The reproducibility question is narrower: does the build run with declared inputs, and can a reviewer inspect how those inputs were selected?

11.5 Design CI as an Independent Recheck

Continuous integration (CI) should reconstruct the project from the checked-out revision and report whether the required checks pass. Its value is independence: CI starts from a defined runner image or hosted environment rather than inheriting your untracked local state. Keep the CI commands aligned with the documented local recipes. If CI contains bespoke build logic, explain why it differs and ensure both paths remain covered.

A useful pipeline separates fast feedback from publication. First check formatting, source structure, dependency consistency, and static rules. Then install or restore dependencies, run tests, build the outputs, and validate the result. For a release, add any required licence, security, provenance, or manual approval checks. Do not make every edit wait for an expensive release-only check, but do not let speed turn a non-blocking signal into an unreviewed release gate.

Make failures legible. Preserve the command, exit status, logs, test reports, and build outputs needed for diagnosis. A shell pipeline can hide an earlier command's failure if it returns only the final command's status; use the shell's pipeline-failure mode or capture each status explicitly. CI should fail when a required step fails, not merely print a red-looking line and continue to upload a success report.

Treat workflow configuration as executable code. Limit token permissions to what each job needs, avoid exposing secrets to jobs that process untrusted pull requests, and pin third-party actions to a full commit SHA when using GitHub Actions. GitHub describes a full SHA as the immutable way to select an action revision in its security hardening guidance. A pin still needs review and updates; it identifies code, not trustworthiness.

Use caches only to save time. A cache key should include the inputs that affect the cached data, such as the operating system and dependency lock. A cache miss must trigger normal reconstruction. Do not use a mutable cache as the only copy of a build result or as evidence that a release was built from a particular commit.

CI is not a substitute for local verification or review. It tests a finite path under a specific configuration. Record what it did, retain useful logs, and inspect warnings that indicate skipped or unsupported checks. If a workflow runs only on the default branch, it cannot validate a pull request before merge; choose

triggers and branch protections that match the intended gate. For a product-specific workflow reference, consult the provider's maintained documentation rather than copying a stale example.

11.6 Treat Generated Artefacts as Published Interfaces

An artefact is a file produced by a build or workflow. It may be a binary, package, report, website, notebook output, model, or the committed HTML and PDF paired with a primer source. Decide whether each generated file is a deliverable, a temporary intermediate, a diagnostic report, or a cache entry. Each category has different retention and review requirements.

If users consume a generated file directly, validate and publish it as a first-class output. Record the source revision and build inputs that produced it, and keep the source and output changes together when the repository's policy requires that pairing. In this repository, rendered primer HTML and PDF are committed because they are the publication target. Mermaid intermediates and ordinary build scratch stay under ignored build directories; the generation script and intended figures are committed when they are part of the document's reproducible production path.

Validation should match the artefact. Check that expected files exist, are non-empty, and can be parsed or opened. Compare metadata such as title, author, version, and page count where relevant. Run link and structural checks over the published HTML, not only its Markdown source, because rendering can change anchors and embedded resources. For a PDF, inspect extracted text and representative pages; a zero exit code cannot reveal clipped content or broken page layout. Hashes can confirm that two files are byte-identical, but only if the build is expected to be deterministic and both hashes refer to the same specified artefact.

Separate validation from provenance. A test result says something about behaviour under its test conditions. A checksum identifies bytes. An attestation can connect an artefact with a repository, workflow, and commit. None alone proves the artefact is correct for its users. GitHub's workflow artefact documentation distinguishes artefacts used to preserve and share outputs from caches used to accelerate reconstruction, and describes attestations as build provenance evidence.

Store transient CI artefacts for the period needed to diagnose or review a run; make long-term retention explicit for release outputs. Name artefacts with enough context to identify the workflow and source revision, and avoid ambiguous names such as `output.zip` in a release archive. When downloading an artefact, verify that it came from the intended run and commit before using or publishing it.

11.7 Freshness, Validation, and the Stale-Output Failure

Freshness is a relation between a source revision and the output built from it. File modification times are weak evidence: archive extraction, clock skew, checkout operations, or copying can alter them. Git history can establish ordering: if the source changed in commit A and the output was last changed in an ancestor of A, the output is stale relative to that source change. This still does not prove that a later output was built correctly; a reviewer can commit an old file after the source edit. Freshness checks are one gate among build, structural, and visual validation.

The following worked example uses the repository's published primer as a model. It changes a source file in a throwaway clone, commits that change without rendering, and runs the history-aware freshness check. Run it from this repository's root, or set `REPOSITORY_PATH` to another checkout. The clone copies real repository history, but all mutations and generated data remain in the disposable copy; the project's container runtime is also required.

```
scratch=$(mktemp -d)
repository_path=${REPOSITORY_PATH:-$PWD}
git clone --no-hardlinks --local "$repository_path" "$scratch/repo"
git -C "$scratch/repo" config user.name 'Disposable Example'
git -C "$scratch/repo" config user.email disposable@example.invalid
printf '\nFreshness probe.\n' >> \
  "$scratch/repo/primer_evaluation_loop/primer_evaluation_loop.md"
git -C "$scratch/repo" add primer_evaluation_loop/primer_evaluation_loop.md
git -C "$scratch/repo" commit -m 'Probe stale published output'
cd "$scratch/repo"
just check-fresh
```

The source commit is newer than the last commits that changed its HTML and PDF, so `check-fresh` reports both outputs as older than their source and exits unsuccessfully. The recovery is to regenerate both outputs with the project's documented container build, inspect the resulting files, and commit them with the source change. Run `just check-fresh` again; then run the relevant render QA and repository checks. Discard the clone when finished. Do not test this by editing the live source and trying to undo the commit afterwards.

This gate deliberately makes a modest claim. It detects the common omission "source changed, committed outputs were not rebuilt". It does not parse the output to prove that it reflects the source, validate its layout, or detect a build that produced the wrong content and was committed later. A strong release combines history or manifest checks with rebuilding and artefact-specific validation. This repository's check documentation documents the freshness recipe and its disposable negative-test method.

When exact byte equality is not a realistic target, define useful semantic checks instead. For a rendered document, those might include expected headings and anchors, no raw Mermaid fences, a valid PDF with extractable text, successful relative links, and human inspection of pages where layout changed. For a compiled package, they might include executable tests, metadata checks, dependency inventories, and a smoke test in a clean runtime. Make the gate explain which property it checks and which properties remain for reviewers.

12 Review, Release, and Safe Lifecycle

The final workflow section connects a reviewable patch to a validated release and its maintenance path. First make the change legible to reviewers; then identify the build, release, recovery, and maintenance states that keep the result trustworthy.

12.1 Repository Hygiene and Reviewable Changes

Keep one coherent change per branch or pull request. A source edit, lockfile update, generated output, and build-recipe change may belong together when they implement one outcome; unrelated formatting or dependency churn makes it harder to establish cause. Inspect the working tree before starting, stage explicit paths, and review the staged diff. Ignore rules should exclude caches, local configuration, credentials, and intermediates without hiding source or required deliverables.

A clean patch is not necessarily a correct patch. Review changes to generated outputs alongside their source. Large generated diffs can be legitimate after a renderer update, but they deserve a representative inspection and a note describing why the output changed. Do not hand-edit a generated file to make it appear fresh: update the source or build process and regenerate it. Preserve

build logs outside the commit unless they are intentional project records.

Commit messages should identify the intent, not merely the operation. Each commit should leave the repository in a state another developer can inspect. Avoid committing secrets, local paths, machine-specific configuration, or outputs produced from untracked source. Before review, verify the branch contains only expected files, inspect whitespace errors, and check whether the documented build and validation commands succeeded.

Reviewers need evidence that connects the change to the result: source diff, dependency changes, commands run, exit status, artefact inspection, and known limitations. A screenshot can help with a visual change, but it does not replace the source or a reproducible build. If a check could not run because the required image or external service was unavailable, state that plainly and provide the strongest independent check that did run.

12.2 Release Workflow and Safe Lifecycle

A safe developer lifecycle is a sequence of explicit states. Start by checking repository instructions, branch state, and the issue or change request. Identify the source, dependencies, build recipe, expected outputs, and acceptance checks before editing. Create an isolated branch or worktree, then make the smallest coherent change. This keeps the work recoverable and makes it possible to compare the result with the starting revision.

Before building, reconstruct the declared environment from a clean state where feasible. Confirm the runtime and container image inputs, restore dependencies from the lockfile, and avoid carrying untracked files into the result. Run focused checks as you work, then the project's full required checks before review. Build every published artefact from the exact source revision intended for review. Inspect the source diff and generated diff together, then validate the artefacts using both automated checks and human inspection where layout or meaning requires it.

Open a review with a concise account of the problem, the change, and the evidence. Identify dependency or image updates, generated outputs, verification commands, and any checks that were not run. Ensure required CI checks correspond to the same commit under review. Respond to review by changing the source of truth, rebuilding affected outputs, and rerunning checks. Avoid push-

ing an artefact from one commit while describing another as its source.

A release candidate should have a stable identity: repository and commit, dependency resolution, tool and image references, build command, and output names or hashes. If the release is built after merge, make the release pipeline build from the merged commit and retain a link or manifest tying outputs to that revision. If an approved artefact is promoted between environments, promote the same bytes rather than rebuilding them under subtly different inputs. Store provenance and validation results beside the release record.

After publication, keep a recovery path. Preserve the prior known-good release and its input record for the retention period the project requires. A rollback should restore a compatible set of source, dependencies, and artefacts; changing only one layer may create a combination that was never validated. If a release is withdrawn or a dependency is found unsafe, identify all outputs that contain or depend on it, stop further distribution, and record what was replaced. For data-producing workflows, retain the manifest and source snapshot needed to explain the result while respecting deletion and privacy requirements.

The lifecycle closes with maintenance. Schedule dependency and base-image updates, check whether the documented build still works from a clean checkout, remove obsolete generated files, and update the workflow when the source tree changes. A pin without an owner becomes stale; a check with no failure response becomes theatre. Give each important input and gate a maintenance route so reproducibility remains a practice rather than a one-time snapshot.

13 Glossary

- **ACL:** an access-control list that adds rules beyond ordinary Unix mode bits.
- **Artefact:** a file produced by a build or workflow, whether it is a published output, report, or temporary result.
- **Branch:** a movable Git reference naming a commit.
- **Child process:** a process started by another process, which may inherit selected attributes from its parent.
- **Environment:** the name-value strings and other external conditions visible to a process.
- **File descriptor:** a process-local integer referring to an open file, pipe, socket, or similar object.

- **Index:** Git's staging area and proposed content for the next commit.
- **Lockfile:** a file recording a selected dependency resolution for repeatable installation.
- **Mount namespace:** a process view of mounted filesystems that can differ from another process's view.
- **Object:** content in Git's object database, such as a blob, tree, or commit.
- **Provenance:** evidence connecting an output to its source, build inputs, and production process.
- **Reference:** a named pointer, such as a branch or tag, that identifies an object.
- **Reflog:** a local log of movements of references such as HEAD and branches.
- **Reproducible build:** a build whose specified inputs produce the claimed level of repeatable output.
- **Shell expansion:** the shell's transformation of parsed words into command arguments.
- **Working tree:** the files checked out for a Git repository at the current HEAD.

14 Further Reading

- Bash Reference Manual — parsing, expansions, redirections, pipelines, status, and command search.
- Linux man-pages project — process, signal, environment, path, mount, and permission interfaces.
- POSIX Shell Command Language — portable shell behaviour and the boundary between POSIX rules and Bash extensions.
- Git documentation — maintained command references; consult the exact page before destructive operations.
- Git Book: Objects and References — conceptual explanation of objects, trees, commits, and names.
- Git reflog documentation — local reference history and recovery considerations.
- Reproducible Builds: Definition — the strict reproducibility claim and its required inputs.
- Repository contribution guide — this repository's build, artefact, freshness,

and render-QA procedures.